PRODUCT ARCHITECTURE

Lexis

An AI-powered daily vocabulary app for iOS that uses on-device Apple Intelligence to surface one genuinely rare, advanced English word each day — completely offline and private.

iOS / SwiftUI · Apple Intelligence · On-Device ML · Server-Driven UI

PRODUCT OUTLINE

How Lexis Works

Lexis is designed around a single interaction loop: one word, one day, one quiz. Every layer of the system — from AI generation to notification scheduling — reinforces this cadence to create a calm, habitual vocabulary practice.

01 AI Word Generation

Each day, Lexis invokes Apple's on-device **FoundationModels** framework (Apple Intelligence) to generate a single advanced vocabulary word. The AI is prompted with strict curation criteria — words must be genuinely obscure and span diverse domains (linguistics, philosophy, natural history, science). A deduplication engine ensures no word is ever repeated.

- Structured JSON prompt → word, IPA phonetic, part of speech, definition, etymology, example sentence
- Generates 3 plausible-but-wrong quiz distractors in the same inference pass
- 3-attempt retry with fallback to curated offline WordBank
- All generation runs on-device — zero network calls, zero data collection

02 Content Processing & Caching

The raw AI output goes through a robust parsing pipeline: markdown fences are stripped, smart quotes are normalized, and the JSON is extracted even if the model adds surrounding prose. Parsed data is encoded into a **WordEntry** model and persisted to UserDefaults so the word loads instantly on revisit — the AI only runs once per day.

- Resilient JSON parser with key-spelling fallbacks (partOfSpeech / part_of_speech)
- Persistent seen-words set prevents lifetime repeats across sessions
- Archive grows daily with full word history

03 User Experience Layer

The word is presented on a dark, literary interface inspired by reading by lamplight. Users see the word in serif typography, hear it pronounced via **AVSpeechSynthesizer**, and explore its definition, etymology, and example sentence in a carefully typeset layout. The UI is driven by a **Server-Driven UI engine** backed by CloudKit, allowing layout updates without app releases.

- Four-tab navigation: Today, Archive, Quiz, Settings
- Text-to-speech pronunciation with British English voice
- Server-Driven UI: CloudKit delivers layout descriptors → ComponentRegistry resolves SwiftUI views
- Streak tracking with visual gold dot indicators

04 AI-Powered Quiz System

At a user-configurable time each day, a quiz unlocks presenting the day's word alongside 4 definition choices — the correct one plus 3 AI-generated distractors. These wrong answers are generated alongside the word in a single inference call, ensuring they are contextually plausible. One attempt per day; results persist and cannot be retried.

- Time-locked quiz with configurable unlock schedule
- AI-generated distractors calibrated to be plausible but incorrect
- Instant visual feedback: green glow for correct, muted red for wrong
- Quiz state persists via UserDefaults with date-based answer tracking

05 Notifications & Habit Loop

Two independent local notification channels — one for the daily word, one for the quiz reminder — are fully configurable with on/off toggles and time pickers. A notification preview shows exactly what appears on the lock screen. This closing of the habit loop (prompt → learn → quiz) drives daily retention.

- UNUserNotifications with repeating calendar triggers
- Independent scheduling for word delivery and quiz reminder
- Preview rendering matches iOS lock screen appearance

06 Privacy-First Analytics

Telemetry is captured via CloudKit's public database — no third-party SDKs, no user accounts, no personally identifiable data. Events track funnel metrics (app launch → word viewed → quiz started → quiz answered), latency measurements for AI generation, and error diagnostics. All analytics calls are fire-and-forget on background threads.

- CloudKit TelemetryEvent records: event name, timestamp, latency, error context
- Async/await with background priority — never blocks UI
- Performance instrumentation wraps AI generation with wall-clock timing

TECHNOLOGY

Stack & Architecture

Lexis is a native SwiftUI app built entirely with Apple-first technologies. No external dependencies beyond AdMob — everything else leverages the platform.

SwiftUI | FoundationModels (Apple Intelligence) | AVSpeechSynthesizer | CloudKit | UserNotifications | UserDefaults | Google AdMob | Server-Driven UI

🧠

On-Device AI

FoundationModels LanguageModelSession generates words entirely on-device with zero network dependency

☁️

CloudKit Backend

Server-Driven UI layouts and privacy-first telemetry via iCloud public database

🔒

Privacy First

No accounts, no tracking, no PII. All AI processing stays on the user's device

DESIGN DECISIONS

Key Technical Choices

Why On-Device AI?

Eliminates API costs, removes network dependency, and ensures complete privacy. The app works fully offline after installation — no server required for core functionality.

Why Server-Driven UI?

CloudKit-backed layout descriptors allow iterating on screen composition, reordering sections, and inserting ad placements without App Store review cycles.

Why Single-Word Cadence?

Cognitive science shows spaced repetition with minimal load outperforms bulk learning. One word per day respects attention and builds lasting habit loops.

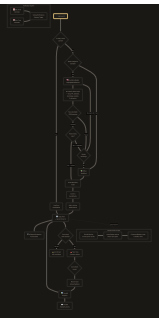
Why Offline WordBank Fallback?

Graceful degradation ensures the app always delivers value — even on devices that don't support Apple Intelligence or when the AI exhausts retries.

SYSTEM FLOW

AI-Powered Workflow Diagram

End-to-end data flow from app launch through AI generation, content delivery, quiz interaction, and analytics capture.



LEXIS

AI-Powered Daily Vocabulary for iOS